# Beaver's Choice Paper Company — Multi-Agent System

## Reflection Report

| | |
|---|---|
| **Framework** | `smolagents` 1.26 |
| **Model** | `gpt-4o-mini` at `temperature=0.1`, via the Vocareum OpenAI-compatible proxy (`https://openai.vocareum.com/v1`) |
| **Agents** | 4 — one orchestrator + three workers, against a permitted maximum of five |
| **Diagram** | `diagram/beavers_choice_workflow.png` |
| **Evaluation output** | `test_results.csv`, full agent trace in `run_full.log` |

## 1. The agent workflow and why it looks like this

### 1.1 The four agents

The system is a strict hierarchy: one orchestrator that owns no tools and never touches the database, and three worker agents that each own one business concern.

| # | Agent | Type | Owns | Explicitly not its job |
|---|---|---|---|---|
| 1 | `orchestrator_agent` | `CodeAgent` | Parsing the request into line items, delegation order, and writing the single customer reply | Never touches the database |
| 2 | `inventory_agent` | `ToolCallingAgent` | Stock truth, supplier lead times, reorder decisions | Never prices, never sells |
| 3 | `quoting_agent` | `ToolCallingAgent` | Pricing and the bulk-discount ladder | Never checks stock, never sells |
| 4 | `sales_agent` | `ToolCallingAgent` | Committing or refusing transactions | Never sets the price |

**Why these particular boundaries.** I did not split the work by "task the customer asks about" but by *failure mode*. A wrong stock reading oversells goods we do not have; a wrong price quietly destroys margin; a wrong database write corrupts the ledger. Those are three different kinds of damage with three different remedies, so each gets one owner. When a response is wrong, the trace in `run_full.log` points at exactly one agent.

**Why `CodeAgent` for the orchestrator and `ToolCallingAgent` for the workers.** Each worker does one narrow job and mostly needs to call a tool and report the result, so structured JSON tool calls are

cheaper, faster and leave less room to improvise. The orchestrator has to loop over several line items per request and branch on each one's availability, so it benefits from `CodeAgent`, which reasons in Python and can hold intermediate values across steps.

**Why four agents and not five.** The rubric permits five and I deliberately used four. The obvious fifth candidate — a "customer communications" agent that turns the workers' findings into prose — would duplicate the orchestrator's final step, add a round-trip of latency and token cost, and give two agents overlapping responsibility. Fewer agents with cleaner boundaries beat more agents with blurred ones.

### 1.2 The decision flow

For each request the orchestrator runs a fixed five-step procedure:

1. **Understand.** Extract each product line with its quantity, and the date the customer needs delivery by. If no date is stated, assume the request date plus `DEFAULT_LEAD_DAYS` (10).
2. **Availability.** Delegate all line items to `inventory_agent` in one call. It returns a per-line verdict of `AVAILABLE`, `RESTOCKED`, or `UNAVAILABLE` with a reason.
3. **Price.** Delegate the resolved item names and quantities to `quoting_agent`, which returns a final total and the discount rate applied.
4. **Fulfil.** Send only the `AVAILABLE` and `RESTOCKED` lines to `sales_agent`. Lines marked `UNAVAILABLE` are never sent, so they cannot be sold by accident. If no line is fulfillable, `sales_agent` is not called at all.
5. **Reply.** Compose one customer-facing message.

The important property is that **the decision to decline is structural, not prompted**. A line is declined because the orchestrator's Python control flow never routes it to `sales_agent`, not because a prompt asked the model to be careful.

### 1.3 The design principle: the LLM decides *which*, Python decides *what*

Every number in the system — the discount tier, the line total, the cash-reserve check, the delivery-deadline check, the stock comparison — is computed in Python inside a tool. The model only chooses which tool to call and with what arguments. It never performs arithmetic and never invents a date.

Business policy therefore lives in named constants, not prose:

```
SUPPLIER_COST_RATIO      = 0.50     # cost of goods, as a fraction of retail
MIN_CASH_RESERVE         = 5_000.00 # a restock may never breach this
BULK_DISCOUNT_TIERS      = [(10_000, 0.15), (5_000, 0.12),
                            (1_000, 0.10), (500, 0.05), (100, 0.02)]
RESTOCK_BUFFER_MULTIPLIER = 1.5     # order the shortfall plus a safety buffer
DEFAULT_LEAD_DAYS         = 10      # assumed deadline when none is stated
```

Two of these are assumptions I imposed on the starter data and should be stated plainly. The starter ships only a retail `unit_price` per item, so `SUPPLIER_COST_RATIO = 0.50` models a 50% cost of goods; changing that single constant models a different margin. `MIN_CASH_RESERVE` is a solvency floor of my own choosing — the company must not spend itself to zero to win one order.

### 1.4 The bug that shaped the architecture

The first working version gave `inventory_agent` separate `check_stock_level` and `place_restock_order` tools and told it, in its instructions, to restock when stock was short. In testing it did something worse than fail: it checked stock, observed a shortfall, called `check_supplier_delivery` — and then **never placed the order**, while the orchestrator went on to quote the customer a price and promise a delivery date for goods that had never been bought.

The fix was not a sterner prompt. It was collapsing check → decide → order into a single atomic tool, `ensure_stock_available`, which resolves the item name, measures the shortfall, sizes the restock, applies both guards, and returns one of three verdicts that the orchestrator branches on:

```
AVAILABLE   — ships from stock
RESTOCKED   — was N units short; M units ordered, arriving <date>, in time
UNAVAILABLE — short and cannot be restocked, because <reason>
```

The generalisable lesson: **when a multi-step decision must always happen, encode it in one tool rather than hoping the model chains the steps.** This is the single most consequential design decision in the project, and it is why the inventory agent's toolbox has one starred primary tool alongside six diagnostic ones.

### 1.5 Handling the starter data as it actually is

Two properties of the provided code shaped the tools:

- **`search_quote_history` joins its search terms with `AND`.** Passing four keywords almost always returns nothing. `find_similar_quotes` therefore retries with progressively fewer terms — all terms, then each term alone — rather than reporting "no precedent found" and letting the model guess.
- **The seeded `inventory` table covers only 18 of the 44 catalog items.** Pricing and the catalog listing therefore read the starter's own `paper_supplies` list rather than the seeded table, because an item we do not currently hold is still an item we can quote and reorder. Stock levels still come from `get_stock_level` against the transactions ledger.

**Item-name resolution is also done in Python**, not by the model (`_resolve_item_name`): exact match → case-insensitive → longest catalog name contained in the customer's phrasing → `difflib` close match. That is how "*heavy cardstock (white)*" resolves to `Cardstock`, and how "*A4 glossy paper*" correctly resolves to `Glossy paper` rather than `A4 paper`.

---

## 2. Tools and helper-function coverage

All seven required helper functions from the starter code are used in at least one tool definition.

| Starter helper | Used by tool(s) | Owning agent(s) |
|---|---|---|
| `create_transaction` | `ensure_stock_available`, `place_restock_order`, `record_sale` | inventory, sales |
| `get_all_inventory` | `inventory_snapshot` | inventory |
| `get_stock_level` | `ensure_stock_available`, `check_stock_level`, `calculate_quote`, `record_sale` | inventory, quoting, sales |
| `get_supplier_delivery_date` | `ensure_stock_available`, `place_restock_order`, `check_supplier_delivery` | inventory, sales |
| `get_cash_balance` | `ensure_stock_available`, `place_restock_order`, `check_cash_balance` | inventory |
| `generate_financial_report` | `company_financial_report` | sales |
| `search_quote_history` | `find_similar_quotes` | quoting |

`get_all_inventory`, `get_cash_balance` and `generate_financial_report` are assigned as internal reporting and health-check tools: `company_financial_report` is run by `sales_agent` before orders above $5,000, and `check_cash_balance` funds the restock decision. Their figures are marked internal in the tool docstrings and are never shown to a customer.

---

## 3. Customer-facing transparency and safety

Four mechanisms keep the customer output complete but not leaky:

1. **Completeness.** The orchestrator's reply specification requires each item and quantity, the total price, the discount rate *and the order size that earned it*, the expected availability date, and — for any declined line — a specific reason.
2. **Justification.** Because `calculate_quote` returns the discount tier alongside the total, the reply can always explain *why* a price is what it is, rather than asserting a number.
3. **Confidentiality.** `QUOTING_AGENT_RULES` forbids mentioning supplier cost, margin, or cash position; `company_financial_report`'s docstring marks its figures internal; the orchestrator is told never to reveal balances, tool output, error messages, or the names of its internal agents.
4. **No leaked failures.** `handle_customer_request` catches every exception, logs the real cause to stdout for us, and returns a neutral message to the customer. A Python traceback can never reach `test_results.csv`.
5. **Money is never authored by the model.** `_enforce_authoritative_pricing` reads back the sales this request actually wrote to the ledger, replaces any money figure in the prose that does not match one of them, and appends an itemised *Confirmed order* block generated from the ledger. See section 4.7.
6. **A deterministic backstop on the vocabulary.** `_sanitize_customer_reply` runs on the final message before it leaves the system, rewriting the internal status codes (`AVAILABLE`, `RESTOCKED`, `UNAVAILABLE`, `SALE COMPLETED`, `SALE REJECTED`) into plain English and logging any remaining

internal term for audit. Rules 1–3 are instructions and therefore advisory; this one is code. See section 4.6.

---

## 4. Evaluation results

The system was run against all 20 requests in `quote_requests_sample.csv`. Results are in `test_results.csv`; the full agent trace is in `run_full.log`. Four scripts reproduce the checks below: `analyze_results.py` (rubric thresholds), `audit_prices.py` (every sale matches its quote) and `audit_replies.py` (no internal vocabulary reaches a customer, and every price shown is backed by a recorded sale).

### 4.1 Run summary

| Metric | Value | Rubric threshold |
|---|---|---|
| Requests processed | 20 / 20 | all |
| Requests that changed the cash balance | **17** | ≥ 3 |
| Requests with at least one recorded sale | **17** | ≥ 3 |
| Requests left entirely unfulfilled | **3** (#13, #15, #19) | ≥ 1 |
| Requests containing at least one declined line | 9 | — |
| Sales priced differently from their quote | **0 / 39** | — |
| Prices shown to a customer not backed by the ledger | **0 / 20** | — |
| Internal status codes leaked to a customer | **0 / 20** | — |
| Runtime errors / fallback responses | 0 | — |

| Financial position | Start | End | Change |
|---|---|---|---|
| Cash balance | $45,059.70 | $46,589.49 | **+$1,529.79** |
| Inventory value | $4,940.30 | $4,740.80 | –$199.50 |
| Total assets | $50,000.00 | $51,330.29 | +$1,330.29 |

Beneath those figures the agents wrote 71 transactions: **39 sales lines** totalling **$4,323.40** across **17 distinct catalog items**, and **32 supplier restock orders** totalling **$2,793.61** — a **35.4% net margin** on the period's trading. A further **14 restock orders were refused by a guard** before any money moved. The company ended more solvent than it began, which is the real test of whether the discount ladder and the cost-of-goods assumption are set sensibly.

### 4.2 Fulfilled orders

**Request 14** (2025-04-09) is the most instructive success because it is a *partial* fulfilment. Two lines sold — 5,000 sheets of A4 paper at a 12% bulk discount ($220.00) and 500 sheets of cardstock at 5% ($71.25) — while the third was declined with a stated reason. The reply closes with a ledger-generated confirmation:

```
Confirmed order
  - 500 x Cardstock - $71.25 (5% bulk discount)
  - 5000 x A4 paper - $220.00 (12% bulk discount)
  Order total: $291.25
```

**Request 17** (2025-04-14) sold two of five lines at $47.50 each and declined three, each with its own reason. This request is worth tracking: it exposed two of the three defects described below, and in this run its customer-facing totals match the ledger exactly.

**Request 20** (2025-04-17) closed the period with three lines totalling $1,365.00 at the 10–15% tiers.

A request's cash delta is *net*: sales minus any restock bought to serve it.

### 4.3 Declined orders

Three requests were declined outright, and each reason traces to a specific code guard rather than a model judgement:

- **Request 15** — "*A4 white paper: could not be fulfilled because not enough stock on hand. A3 colored paper: could not be fulfilled because not enough stock on hand. cardboard for signage: could not be fulfilled because the item is not one we carry.*" The first two are the `_restock` delivery-deadline guard; the third is `_resolve_item_name` correctly refusing to force a match onto a real product.
- **Request 19** — three lines critically short, none restockable before the 20 April deadline.
- **Request 13** — cardstock short, restock arriving after the deadline.

### 4.4 Strengths

1. **Quote and ledger cannot diverge.** All 39 recorded sales match the price the customer was quoted ( `audit_prices.py` ).
2. **Every price a customer sees is backed by a recorded sale** ( `audit_replies.py` ), enforced by code rather than by instruction.
3. **No internal vocabulary reaches customers**, verified across all 20 replies.
4. **The system refuses rather than overselling.** No transaction was written for stock that did not exist.
5. **Refusals are specific and actionable** — units short, supplier delivery against the deadline, or an item not carried.
6. **Partial fulfilment works.** The system sells what it can and declines the rest in one coherent reply.
7. **Trading was profitable and solvent.** Cash never approached the $5,000 reserve; the period closed $1,529.79 up.
8. **Zero failures.** No request hit the exception fallback.

### 4.5 First defect: the ledger trusted the model

An early full run passed every rubric threshold and closed $1,899.57 up — and was wrong. Auditing each recorded sale against the price the customer had been quoted revealed **3 of 40 sale lines billed at a**

**price the customer was never quoted**: two overcharged by $190 each (a $47.50 line written to the ledger at $237.50) and one undercharged — a net overcharge of $356.48.

`calculate_quote` computed the price deterministically, but `record_sale` accepted `total_price` as an argument and wrote whatever it was given, so the agreed figure had to survive a hop from `quoting_agent` through the orchestrator to `sales_agent`.

`record_sale` now **re-derives** the line total from the catalog price and the discount ladder, treats the caller's figure as a claim, and logs any disagreement as a `[price correction]`. In a later run that guard fired three times in a single request, once against a claimed $1,125.00 on a line worth $5.50 — so the corruption is real and recurrent, not a one-off.

### 4.6 Second defect: internal vocabulary leaked to a customer

The next run priced the ledger correctly but leaked differently: request 17 rendered its declined lines as "*A4 white printer paper: UNAVAILABLE*", passing an internal verdict token to the customer three times in one message.

The fix has two layers. The orchestrator's rules now name the five status codes as internal and give the phrasing to use instead; and `_sanitize_customer_reply` rewrites any surviving all-capitals token before the message leaves `handle_customer_request`, logging any other internal term for audit.

### 4.7 Third defect: the reply was priced by the model, not the ledger

Fixing the ledger did not fix the customer. In the run after 4.5, request 17's reply told the customer "*$237.50 … $237.50 … total $475.00*" while the ledger, correctly, charged **$95.00**. The trace shows exactly what happened: `calculate_quote` returned $47.50 for each line; the model corrupted it to $237.50 on the hop to `sales_agent`; `record_sale`'s guard caught it and charged $47.50 — and the orchestrator then wrote the *corrupted* figure into the reply.

The guard had protected the ledger and left the customer-facing message exposed. `audit_prices.py` reported clean because it only ever checked the ledger.

The fix makes money in the reply come from the ledger too. `handle_customer_request` marks the ledger position before the run, reads back exactly the sales that run produced, replaces any money figure in the prose that does not match one of them with an em dash, and appends an itemised *Confirmed order* block generated from the ledger. Correct figures pass through untouched; a wrong one is removed. `audit_replies.py` was extended to verify the property, and reports zero unbacked prices across all 20 replies.

In this run the guard intervened once, in request 8, where the model's combined total was wrong while all four line prices were right: the bad total was removed and the ledger's `$653.00` shown instead.

**The pattern across all three defects.** Each was caused by trusting the model to carry a value faithfully across a hop, and each was fixed by moving the guarantee into code at the point of use. Stated once: *a value that must be correct should be computed where it is used, not passed between agents.* The architecture in section 1.4 came from the same realisation.

### 4.8 Remaining weaknesses

1. **The prose and the ledger block can disagree in tone.** When the guard fires, the customer sees an em dash mid-sentence followed by a correct summary. It is never wrong, but it is not elegant — the real fix is 5.2.
2. **Replenishment is purely reactive.** Stock is only ordered once a customer has asked for it, by which point the supplier lead time (4 days over 100 units, 7 days over 1,000) frequently exceeds the deadline. Inventory value *fell* over the period despite 32 restock orders, and 14 further restocks were refused on timing. This accounts for every outright decline.
3. **Stock is occasionally bought for an order that is then declined**, because the restock decision is made per line before the orchestrator knows whether the order as a whole will proceed.
4. **Two copies of the pricing formula.** `calculate_quote` and `record_sale` each compute the line total independently. They agree today and `audit_prices.py` proves it, but a change to one without the other would silently reintroduce 4.5.
5. **Cost and latency.** Four agents and several LLM calls per request meant roughly 2–3 minutes per request, about 50 minutes for a full run.

---

## 5. Suggestions for further improvement

### 5.1 Replace reactive restocking with a predictive reorder policy

`ensure_stock_available` only buys stock after a customer has asked for something we do not have — by which time the supplier lead time usually exceeds their deadline. All three outright declines and most partial ones trace to this.

The starter data already contains what is needed: the `inventory` table carries `min_stock_level` per item, and `transactions` gives sales velocity. A scheduled reorder pass — run between requests, or as a fifth agent using the one remaining slot in the five-agent budget — could reorder any item whose projected stock at the end of its lead time falls below `min_stock_level`. Because the declines were caused by *timing* rather than by price or solvency, this alone would convert declined revenue into sales without touching the pricing logic, and would largely dissolve weakness 4.8.3 as a side effect.

### 5.2 Compose the whole reply from a template, not just the prices

Section 4.7 fixed pricing by generating it from the ledger, but the surrounding prose is still authored by the model, which is why a corrected reply can read awkwardly. The same treatment should extend to the whole message.

`handle_customer_request` would assemble the reply from a structured result — a list of `(item, quantity, verdict, total, discount_rate, availability_date, decline_reason)` records rendered through a Python template — and assert that the number of verdicts returned equals the number of line items requested, re-querying the inventory agent for any line missing one. The model would still do the language understanding at the front of the pipeline and the judgement in the middle; it would simply stop being the last thing between the data and the customer. Every defect in 4.5–4.7 would then be structurally impossible rather than caught after the fact.

### 5.3 Factor the pricing formula into one shared function, and make it margin-aware

Two improvements to the same code. First, weakness 4.8.4: extract the shared computation into a single `_line_total(item_name, quantity)` used by both `calculate_quote` and `record_sale`, so quote and ledger are identical by construction rather than by audit.

Second, `BULK_DISCOUNT_TIERS` is a flat function of quantity, applied identically to a $0.02 napkin and a $2.50 roll of banner paper. At the assumed 50% cost of goods a 15% discount is comfortably absorbed, but the ladder has no knowledge of that and would keep discounting if `SUPPLIER_COST_RATIO` were raised or per-item supplier costs introduced. A margin-aware `_line_total` would compute the implied margin after discount and cap it at whatever keeps the line above a floor — say 20% gross. The observed 35.4% margin is healthy by construction rather than by control; nothing currently prevents an unprofitable quote. The floor would also let the company offer *deeper* discounts on genuinely high-margin items to win larger orders.

---

## 6. Files submitted

| File | Contents |
|---|---|
| `diagram/beavers_choice_workflow.png` | Agent workflow diagram — 4 agents, 14 tool bindings, each labelled with its purpose and the starter helper function it wraps |
| `project_starter.py` | Complete implementation in a single Python file |
| `test_results.csv` | Evaluation output over all 20 sample requests |
| `report/reflection_report.pdf` | This report |

Supporting evidence, not part of the required submission: `run_full.log` (full agent trace), the four verification scripts (`analyze_results.py`, `audit_prices.py`, `audit_replies.py`), and two earlier runs kept as before-pictures — `run_prev_with_leak.log` for section 4.6 and `run_prev_reply_price_bug.log` for section 4.7, with their `test_results` CSVs.